```java
package Grafos;
import java.util.ArrayList;

public class Grafo implements InterfaceGrafo{
    private Vertice[] vertices;
    private ArrayList<Aresta>[][] arestas;
    private int capacidade, size;

    public Grafo(int capacidade) {
        this.capacidade = capacidade;
        this.size = 0;
        this.vertices = new Vertice[capacidade];
        this.arestas = new ArrayList[capacidade][capacidade];

        //Por ser uma matriz de arestas, então precisa definir o limite pela capacidade
        for (int i = 0; i < capacidade; i++){
            for (int j = 0; j < capacidade; j++){
                this.arestas[i][j] = new ArrayList<Aresta>();
            }
        }

    }

    public void removeVertice(Vertice v) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice não encontrado");
        }

        for (int i = 0; i < vertices.length; i++){
            if (vertices[i] == v){

                //preciso tirar aresta que nao vai ter mais ligacao
                for (int j = 0; j < arestas.length; j++){
                    arestas[i][j].clear();
                    arestas[j][i].clear();
                }

                vertices[i] = null;
                size--;
                return;
            }

        }

        throw new Correcao("Vertice não encontrado");
    }

    public void removeAresta(Aresta a) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        boolean encontrouAresta = false;

        //verifica a existencia da aresta no grafo, caso exista, remove a aresta
        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
```

```
62                  if (arestas[i][j].remove(a)){
63                      encontrouAresta = true;
64                  }
65              }
66          }
67
68          if (!encontrouAresta){
69              throw new Correcao("Aresta não encontrada");
70          }
71      }
72
73      public void insertVertice(Vertice v) throws Correcao {
74          if (v == null){
75              throw new Correcao("Vertice não encontrado");
76          }
77
78          //vertice entra no grafo e vê um vazio para ser colocado na lista
79          for (int i = 0; i < vertices.length; i++){
80              if (vertices[i] == null){
81                  vertices[i] = v;
82                  size++;
83                  return;
84              }
85          }
86
87          throw new Correcao("Grafo cheio");
88
89      }
90
91      public void insertAresta(Aresta a, Vertice v1, Vertice v2) throws Correcao {
92          if (isEmpty()){
93              throw new Correcao("Grafo vazio");
94          }
95
96          if (v1 == null || v2 == null){
97              throw new Correcao("Vertice não encontrado");
98          }
99
100         if (a == null){
101             throw new Correcao("Aresta não encontrada");
102         }
103
104         for (int i = 0; i < arestas.length; i++){
105             for (int j = 0; j < arestas[i].length; j++){
106                 if (vertices[i] == v1 && vertices[j] == v2){
107
108                     //precisa definir origem e fim
109                     a.setOrigem(v1);
110                     a.setFim(v2);
111
112                     if (a.getDirecionada() == true){
113                         arestas[i][j].add(a);
114                         return;
115                     }
116
117                     //sao do mesmo sentido
118                     else{
119                         arestas[i][j].add(a);
120
121                         //se for laço (i == j), não pode adicionar duas vezes na mesma célula
122                         if (i != j){
123                             arestas[j][i].add(a);
124                         }
```

```
125                         return;
126                     }
127                 }
128             }
129         }
130 
131         throw new Correcao("Vertices não encontrados");
132     }
133 
134     public boolean isAdjacent(Vertice v1, Vertice v2, Aresta a) throws Correcao {
135         if (isEmpty()){
136             throw new Correcao("Grafo vazio");
137         }
138 
139         if (v1 == null || v2 == null){
140             throw new Correcao("Vertice não encontrado");
141         }
142 
143         if (a == null){
144             throw new Correcao("Aresta não encontrada");
145         }
146 
147         for (int i = 0; i < arestas.length; i++){
148             for (int j = 0; j < arestas[i].length; j++){
149                 if (vertices[i] == v1 && vertices[j] == v2){
150                     if (arestas[i][j].contains(a)){
151                         return true;
152                     } else {
153                         return false;
154                     }
155                 }
156             }
157         }
158 
159         throw new Correcao("Vertices não encontrados");
160     }
161 
162     public boolean isEmpty() {
163         return size == 0;
164     }
165 
166     public Vertice oposto(Vertice v, Aresta a) throws Correcao {
167         if (isEmpty()){
168             throw new Correcao("Grafo vazio");
169         }
170 
171         if (v == null){
172             throw new Correcao("Vertice não encontrado");
173         }
174 
175         if (a == null){
176             throw new Correcao("Aresta não encontrada");
177         }
178 
179         if (a.getOrigem() == v){
180             return a.getFim();
181 
182         }
183 
184         else if (a.getFim() == v){
185             return a.getOrigem();
186         }
187 
```

```
188         else {
189             throw new Correcao("Vertice não é oposto da aresta");
190         }
191 
192     }
193 
194     public Vertice[] finalVertices(Aresta a) throws Correcao {
195         if (isEmpty()){
196             throw new Correcao("Grafo vazio");
197         }
198 
199         if (a == null){
200             throw new Correcao("Aresta não encontrada");
201         }
202 
203         //ele retorna um array de vertice que é seu origem e fim que pertencem a mesma aresta
204         for (int i = 0; i < arestas.length; i++){
205             for (int j = 0; j < arestas[i].length; j++){
206                 if (arestas[i][j].contains(a)){
207                     return new Vertice[]{a.getOrigem(), a.getFim()};
208                 }
209             }
210         }
211 
212         throw new Correcao("Aresta não encontrada");
213 
214     }
215 
216     public void substituirVertice(Vertice v, Object elemento) throws Correcao {
217         if (isEmpty()){
218             throw new Correcao("Grafo vazio");
219         }
220 
221         if (v == null){
222             throw new Correcao("Vertice não encontrado");
223         }
224 
225         for (int i = 0; i < vertices.length; i++){
226             if (vertices[i] == v){
227                 vertices[i].setElement(elemento);
228                 return;
229             }
230         }
231 
232         throw new Correcao("Vertice não encontrado");
233     }
234 
235     public void substituirAresta(Aresta a, Object elemento) throws Correcao {
236         if (isEmpty()){
237             throw new Correcao("Grafo vazio");
238         }
239 
240         if (a == null){
241             throw new Correcao("Aresta não encontrada");
242         }
243 
244         for (int i = 0; i < arestas.length; i++){
245             for (int j = 0; j < arestas[i].length; j++){
246                 if (arestas[i][j].contains(a)){
247                     a.setElement(elemento);
248                     return;
249                 }
250             }
```

```
251         }
252 
253         throw new Correcao("Aresta não encontrada");
254 
255     }
256 
257     public boolean isDirected(Aresta a) throws Correcao{
258         if (isEmpty()){
259             throw new Correcao("Grafo vazio");
260         }
261 
262         if (a == null){
263             throw new Correcao("Aresta não encontrada");
264         }
265 
266         for (int i = 0; i < arestas.length; i++){
267             for (int j = 0; j < arestas[i].length; j++){
268                 if (arestas[i][j].contains(a)){
269                     return a.getDirecionada();
270                 }
271             }
272         }
273 
274         throw new Correcao("Aresta não encontrada");
275     }
276 
277     public void insertArestaDirecionada(Aresta a, Vertice v1, Vertice v2) throws Correcao{
278         if (isEmpty()){
279             throw new Correcao("Grafo vazio");
280         }
281 
282         if (a == null){
283             throw new Correcao("Aresta não encontrada");
284         }
285 
286         if (v1 == null || v2 == null){
287             throw new Correcao("Vertice não encontrado");
288         }
289 
290         for (int i = 0; i < arestas.length; i++){
291             for (int j = 0; j < arestas[i].length; j++){
292                 if (vertices[i] == v1 && vertices[j] == v2){
293 
294                     //só altera a aresta depois de confirmar que os vertices existem
295                     a.setDirecionada(true);
296                     a.setOrigem(v1);
297                     a.setFim(v2);
298 
299                     arestas[i][j].add(a);
300                     return;
301                 }
302             }
303         }
304 
305 
306         throw new Correcao("Vertices não encontrados");
307     }
308 
309     public Vertice[] vertices() throws Correcao{
310         if (isEmpty()){
311             throw new Correcao("Grafo vazio");
312         }
313 
```

```
314         return vertices;
315     }
316 
317     public ArrayList<Aresta>[][] arestas() throws Correcao{
318         if (isEmpty()){
319             throw new Correcao("Grafo vazio");
320         }
321 
322         return arestas;
323     }
324 
325     public int grau(Vertice v) throws Correcao{
326         if (isEmpty()){
327             throw new Correcao("Grafo vazio");
328         }
329 
330         if (v == null){
331             throw new Correcao("Vertice não encontrado");
332         }
333 
334         int grau = 0;
335         ArrayList<Aresta> contadas = new ArrayList<Aresta>();
336 
337         for (int i = 0; i < arestas.length; i++){
338             for (int j = 0; j < arestas[i].length; j++){
339                 for (Aresta a : arestas[i][j]){
340                     if (v == a.getOrigem() || v == a.getFim()){
341                         if (!contadas.contains(a)){
342 
343                             //laço conta 2 no grau
344                             if (a.getOrigem() == a.getFim()){
345                                 grau += 2;
346                             } else {
347                                 grau++;
348                             }
349                             contadas.add(a);
350                         }
351 
352                     }
353                 }
354             }
355         }
356 
357         return grau;
358     }
359 
360     public int grauEntrada(Vertice v) throws Correcao{
361         if (isEmpty()){
362             throw new Correcao("Grafo vazio");
363         }
364 
365         if (v == null){
366             throw new Correcao("Vertice não encontrado");
367         }
368 
369         int grauEntrada = 0;
370 
371         for (int i = 0; i < arestas.length; i++){
372             for (int j = 0; j < arestas[i].length; j++){
373                 for (Aresta a : arestas[i][j]){
374 
375                     if (v == a.getFim() && a.getDirecionada() == true){
376                         grauEntrada++;
```

```
377
378                     }
379                 }
380             }
381         }
382
383         return grauEntrada;
384     }
385
386     public int grauSaida(Vertice v) throws Correcao{
387         if (isEmpty()){
388             throw new Correcao("Grafo vazio");
389         }
390
391         if (v == null){
392             throw new Correcao("Vertice não encontrado");
393         }
394
395         int grauSaida = 0;
396
397         for (int i = 0; i < arestas.length; i++){
398             for (int j = 0; j < arestas[i].length; j++){
399                 for (Aresta a : arestas[i][j]){
400                     if (a.getDirecionada() == true && v == a.getOrigem()){
401                         grauSaida++;
402                     }
403                 }
404             }
405         }
406
407         return grauSaida;
408     }
409
410     public ArrayList<Aresta> arestasIncidentes(Vertice v) throws Correcao{
411         if (isEmpty()){
412             throw new Correcao("Grafo vazio");
413         }
414
415         if (v == null){
416             throw new Correcao("Vertice não encontrado");
417         }
418
419         ArrayList<Aresta> contadas = new ArrayList<Aresta>();
420
421         for (int i = 0; i < arestas.length; i++){
422             for (int j = 0; j < arestas[i].length; j++){
423                 for (Aresta a : arestas[i][j]){
424                     if (v == a.getOrigem() || v == a.getFim()){
425                         if (!contadas.contains(a)){
426                             contadas.add(a);
427                         }
428                     }
429                 }
430             }
431         }
432
433         return contadas;
434     }
435
436 }
```